

Компьютерная графика

Лекция 4: Камера, параллельная/ортографическая и перспективная проекции, буфер глубины

2026

- В большинстве ситуаций виртуальный мир – двухмерный/трёхмерный, произвольного размера, наблюдаемый с произвольного ракурса

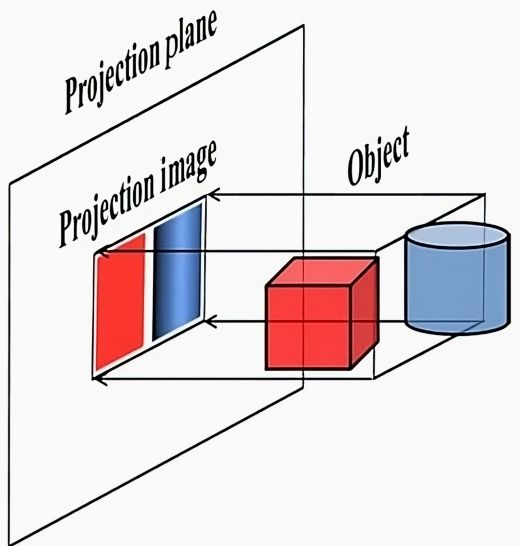
- В большинстве ситуаций виртуальный мир – двухмерный/трёхмерный, произвольного размера, наблюдаемый с произвольного ракурса
- Экран – двухмерный, в системе координат WebGL $[-1, 1] \times [-1, 1]$

- В большинстве ситуаций виртуальный мир – двухмерный/трёхмерный, произвольного размера, наблюдаемый с произвольного ракурса
- Экран – двухмерный, в системе координат WebGPU $[-1, 1] \times [-1, 1]$
 - Координата Z в OpenGL – $[-1, 1]$, в остальных API (включая WebGPU) – $[0, 1]$
 - Про неё поговорим подробнее чуть позже

- В большинстве ситуаций виртуальный мир – двухмерный/трёхмерный, произвольного размера, наблюдаемый с произвольного ракурса
- Экран – двухмерный, в системе координат WebGPU $[-1, 1] \times [-1, 1]$
 - Координата Z в OpenGL – $[-1, 1]$, в остальных API (включая WebGPU) – $[0, 1]$
 - Про неё поговорим подробнее чуть позже
- $\implies$ Нам нужно преобразование из первого во второе, т.н. *проекция*

- В большинстве ситуаций виртуальный мир – двухмерный/трёхмерный, произвольного размера, наблюдаемый с произвольного ракурса
- Экран – двухмерный, в системе координат WebGL $[-1, 1] \times [-1, 1]$
 - Координата Z в OpenGL – $[-1, 1]$, в остальных API (включая WebGL) – $[0, 1]$
 - Про неё поговорим подробнее чуть позже
- $\implies$ Нам нужно преобразование из первого во второе, т.н. *проекция*
- Два основных вида проекции:
 - Параллельная проекция
 - Перспективная проекция

Параллельная проекция



Параллельная проекция

- Самый простой способ проекции:

Параллельная проекция

- Самый простой способ проекции: игнорировать третью координату

Концептуально: $(x, y, z) \mapsto (x, y)$

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Параллельная проекция

- Самый простой способ проекции: игнорировать третью координату

Концептуально: $(x, y, z) \mapsto (x, y)$

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- Именно это делает видеокарта (если `position.w = 1`)

Параллельная проекция

- Самый простой способ проекции: игнорировать третью координату

Концептуально: $(x, y, z) \mapsto (x, y)$

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- Именно это делает видеокарта (если `position.w = 1`)
- X и Y всё ещё $[-1, 1]$

Параллельная проекция

- Самый простой способ проекции: игнорировать третью координату

Концептуально: $(x, y, z) \mapsto (x, y)$

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- Именно это делает видеокарта (если `position.w = 1`)
- X и Y всё ещё $[-1, 1]$
- Что, если $X \in [-W, W]$ и $Y \in [-H, H]$, и нам нужно их перевести в стандартные для WebGL $[-1, 1]^2$?

Параллельная проекция

- Самый простой способ проекции: игнорировать третью координату

Концептуально: $(x, y, z) \mapsto (x, y)$

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- Именно это делает видеокарта (если `position.w = 1`)
- X и Y всё ещё $[-1, 1]$
- Что, если $X \in [-W, W]$ и $Y \in [-H, H]$, и нам нужно их перевести в стандартные для WebGL $[-1, 1]^2$?

$$\begin{pmatrix} \frac{1}{W} & 0 & 0 & 0 \\ 0 & \frac{1}{H} & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- Что, если $X \in [X_0 - W, X_0 + W]$ и $Y \in [Y_0 - H, Y_0 + H]$?

Параллельная проекция

- Что, если $X \in [X_0 - W, X_0 + W]$ и $Y \in [Y_0 - H, Y_0 + H]$?
 - Сдвинуть на $(-X_0, -Y_0)$, а затем применить масштабирование:

$$\begin{pmatrix} \frac{1}{W} & 0 & 0 & 0 \\ 0 & \frac{1}{H} & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} 1 & 0 & 0 & -X_0 \\ 0 & 1 & 0 & -Y_0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Параллельная проекция

- Что, если $X \in [X_0 - W, X_0 + W]$ и $Y \in [Y_0 - H, Y_0 + H]$?
 - Сдвинуть на $(-X_0, -Y_0)$, а затем применить масштабирование:

$$\begin{pmatrix} \frac{1}{W} & 0 & 0 & 0 \\ 0 & \frac{1}{H} & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} 1 & 0 & 0 & -X_0 \\ 0 & 1 & 0 & -Y_0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- Если размер области W , нужно разделить на W

Параллельная проекция

- Что, если $X \in [X_0 - W, X_0 + W]$ и $Y \in [Y_0 - H, Y_0 + H]$?
 - Сдвинуть на $(-X_0, -Y_0)$, а затем применить масштабирование:

$$\begin{pmatrix} \frac{1}{W} & 0 & 0 & 0 \\ 0 & \frac{1}{H} & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} 1 & 0 & 0 & -X_0 \\ 0 & 1 & 0 & -Y_0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- Если размер области W , нужно разделить на W
- Если центр в X_0 , нужно сдвинуть на $-X_0$

Параллельная проекция

- Что, если $X \in [X_0 - W, X_0 + W]$ и $Y \in [Y_0 - H, Y_0 + H]$?
 - Сдвинуть на $(-X_0, -Y_0)$, а затем применить масштабирование:

$$\begin{pmatrix} \frac{1}{W} & 0 & 0 & 0 \\ 0 & \frac{1}{H} & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} 1 & 0 & 0 & -X_0 \\ 0 & 1 & 0 & -Y_0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- Если размер области W , нужно разделить на W
- Если центр в X_0 , нужно сдвинуть на $-X_0$
- $\implies$ Общая идея: если камера получена каким-то преобразованием, нужно применить *обратное преобразование*

Параллельная проекция: обратное преобразование

- Можно считать, что по умолчанию (если `position.w = 1`) видеокарта делает параллельную проекцию на квадрат $[-1, 1]^2$ в плоскости XY параллельно оси Z

Параллельная проекция: обратное преобразование

- Можно считать, что по умолчанию (если `position.w = 1`) видеокарта делает параллельную проекцию на квадрат $[-1, 1]^2$ в плоскости XY параллельно оси Z
- Камера находится в начале координат и никак явным образом не настраивается

Параллельная проекция: обратное преобразование

- Можно считать, что по умолчанию (если `position.w = 1`) видеокарта делает параллельную проекцию на квадрат $[-1, 1]^2$ в плоскости XY параллельно оси Z
- Камера находится в начале координат и никак явным образом не настраивается
- Если ко всему виртуальному миру (включая камеру!) применить аффинное преобразование, изображение не изменится

Параллельная проекция: обратное преобразование

- Можно считать, что по умолчанию (если `position.w = 1`) видеокарта делает параллельную проекцию на квадрат $[-1, 1]^2$ в плоскости XY параллельно оси Z
- Камера находится в начале координат и никак явным образом не настраивается
- Если ко всему виртуальному миру (включая камеру!) применить аффинное преобразование, изображение не изменится
- $\implies$ Если наша камера получена аффинным преобразованием из стандартной камеры WebGPU, к объектам мира нужно применить обратное преобразование

Параллельная проекция: общий случай

- В общем случае параллельную проекцию можно задать

Параллельная проекция: общий случай

- В общем случае параллельную проекцию можно задать
 - Положением камеры $C = (C_x, C_y, C_z)$

Параллельная проекция: общий случай

- В общем случае параллельную проекцию можно задать
 - Положением камеры $C = (C_x, C_y, C_z)$
 - Осями координат камеры
 $X = (X_x, X_y, X_z), Y = (Y_x, Y_y, Y_z), Z = (Z_x, Z_y, Z_z)$

Параллельная проекция: общий случай

- В общем случае параллельную проекцию можно задать
 - Положением камеры $C = (C_x, C_y, C_z)$
 - Осями координат камеры
 $X = (X_x, X_y, X_z), Y = (Y_x, Y_y, Y_z), Z = (Z_x, Z_y, Z_z)$
- Делает проекцию параллельно вектору Z на параллелограмм $C \pm X \pm Y$, который отождествляется с экраном

Параллельная проекция: общий случай

- В общем случае параллельную проекцию можно задать
 - Положением камеры $C = (C_x, C_y, C_z)$
 - Осями координат камеры
 $X = (X_x, X_y, X_z), Y = (Y_x, Y_y, Y_z), Z = (Z_x, Z_y, Z_z)$
- Делает проекцию параллельно вектору Z на параллелограмм $C \pm X \pm Y$, который отождествляется с экраном
- Обычно X, Y, Z взаимно ортогональны, тогда эту проекцию называют *ортографической*

Параллельная проекция: общий случай

- В общем случае параллельную проекцию можно задать
 - Положением камеры $C = (C_x, C_y, C_z)$
 - Осями координат камеры
 $X = (X_x, X_y, X_z), Y = (Y_x, Y_y, Y_z), Z = (Z_x, Z_y, Z_z)$
- Делает проекцию параллельно вектору Z на параллелограмм $C \pm X \pm Y$, который отождествляется с экраном
- Обычно X, Y, Z взаимно ортогональны, тогда эту проекцию называют *ортографической*
- Неортографические параллельные проекции в графике обычно не используют

Параллельная проекция: общий случай

- Как выразить эту проекцию матрицей?

Параллельная проекция: общий случай

- Как выразить эту проекцию матрицей?
- Преобразование из стандартной системы координат WebGPU $[-1, 1] \times [-1, 1] \times [0, 1]$ в координаты области, видимой через эту камеру: поворот системы координат + параллельный перенос

$$\begin{pmatrix} X_x & Y_x & Z_x & C_x \\ X_y & Y_y & Z_y & C_y \\ X_z & Y_z & Z_z & C_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Параллельная проекция: общий случай

- Как выразить эту проекцию матрицей?
- Преобразование из стандартной системы координат WebGPU $[-1, 1] \times [-1, 1] \times [0, 1]$ в координаты области, видимой через эту камеру: поворот системы координат + параллельный перенос

$$\begin{pmatrix} X_x & Y_x & Z_x & C_x \\ X_y & Y_y & Z_y & C_y \\ X_z & Y_z & Z_z & C_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- Обратное преобразование (матрица проекции) – матрица, обратная к этой

Параллельная проекция: общий случай

- Можно представить X, Y, Z как произведение длины и нормированного вектора:

$$X = W \cdot \hat{X} \quad Y = H \cdot \hat{Y} \quad Z = D \cdot \hat{Z}$$

Параллельная проекция: общий случай

- Можно представить X, Y, Z как произведение длины и нормированного вектора:

$$X = W \cdot \hat{X} \quad Y = H \cdot \hat{Y} \quad Z = D \cdot \hat{Z}$$

- Матрицу можно разбить на масштабирование, поворот и перенос:

$$\begin{pmatrix} X_x & Y_x & Z_x & C_x \\ X_y & Y_y & Z_y & C_y \\ X_z & Y_z & Z_z & C_z \\ 0 & 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & C_x \\ 0 & 1 & 0 & C_y \\ 0 & 0 & 1 & C_z \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} \hat{X}_x & \hat{Y}_x & \hat{Z}_x & 0 \\ \hat{X}_y & \hat{Y}_y & \hat{Z}_y & 0 \\ \hat{X}_z & \hat{Y}_z & \hat{Z}_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} W & 0 & 0 & 0 \\ 0 & H & 0 & 0 \\ 0 & 0 & D & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Параллельная проекция: общий случай

- Тогда обратная матрица (матрица проекции):

$$\begin{pmatrix} X_x & Y_x & Z_x & C_x \\ X_y & Y_y & Z_y & C_y \\ X_z & Y_z & Z_z & C_z \\ 0 & 0 & 0 & 1 \end{pmatrix}^{-1} = \begin{pmatrix} \frac{1}{W} & 0 & 0 & 0 \\ 0 & \frac{1}{H} & 0 & 0 \\ 0 & 0 & \frac{1}{D} & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} \hat{X}_x & \hat{Y}_x & \hat{Z}_x & 0 \\ \hat{X}_y & \hat{Y}_y & \hat{Z}_y & 0 \\ \hat{X}_z & \hat{Y}_z & \hat{Z}_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}^{-1} \cdot \begin{pmatrix} 1 & 0 & 0 & -C_x \\ 0 & 1 & 0 & -C_y \\ 0 & 0 & 1 & -C_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Параллельная проекция: общий случай

- Тогда обратная матрица (матрица проекции):

$$\begin{pmatrix} X_x & Y_x & Z_x & C_x \\ X_y & Y_y & Z_y & C_y \\ X_z & Y_z & Z_z & C_z \\ 0 & 0 & 0 & 1 \end{pmatrix}^{-1} = \begin{pmatrix} \frac{1}{W} & 0 & 0 & 0 \\ 0 & \frac{1}{H} & 0 & 0 \\ 0 & 0 & \frac{1}{D} & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} \hat{X}_x & \hat{Y}_x & \hat{Z}_x & 0 \\ \hat{X}_y & \hat{Y}_y & \hat{Z}_y & 0 \\ \hat{X}_z & \hat{Y}_z & \hat{Z}_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}^{-1} \cdot \begin{pmatrix} 1 & 0 & 0 & -C_x \\ 0 & 1 & 0 & -C_y \\ 0 & 0 & 1 & -C_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

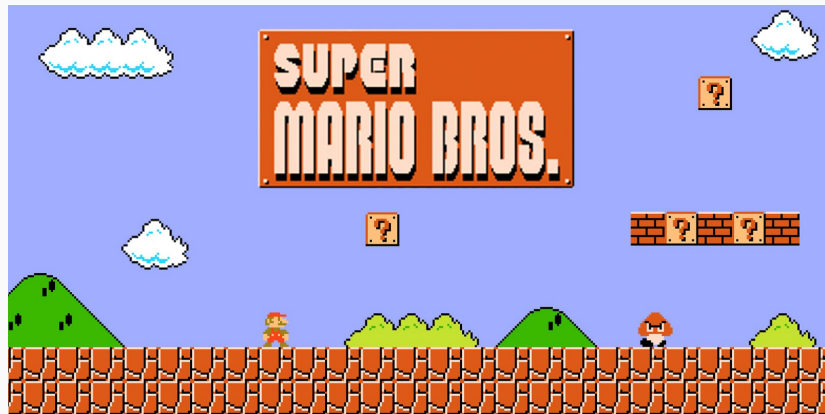
- Эту матрицу мы передаём в шейдер, применяем к входным вершинам, и записываем результат в `@builtin(position)`

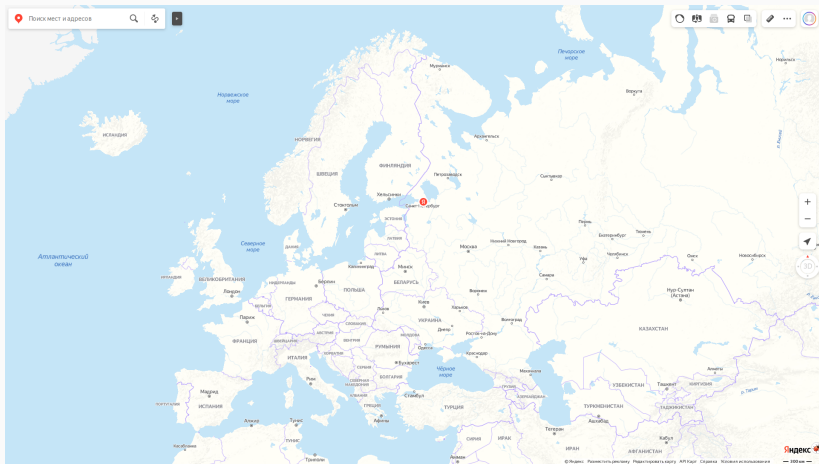
Параллельная проекция: ортогональный случай

- Если X, Y, Z ортогональны (т.е. проекция ортографическая), то

$$\begin{pmatrix} \hat{X}_x & \hat{Y}_x & \hat{Z}_x & 0 \\ \hat{X}_y & \hat{Y}_y & \hat{Z}_y & 0 \\ \hat{X}_z & \hat{Y}_z & \hat{Z}_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}^{-1} = \begin{pmatrix} \hat{X}_x & \hat{Y}_x & \hat{Z}_x & 0 \\ \hat{X}_y & \hat{Y}_y & \hat{Z}_y & 0 \\ \hat{X}_z & \hat{Y}_z & \hat{Z}_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}^T = \begin{pmatrix} \hat{X}_x & \hat{X}_y & \hat{X}_z & 0 \\ \hat{Y}_x & \hat{Y}_y & \hat{Y}_z & 0 \\ \hat{Z}_x & \hat{Z}_y & \hat{Z}_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

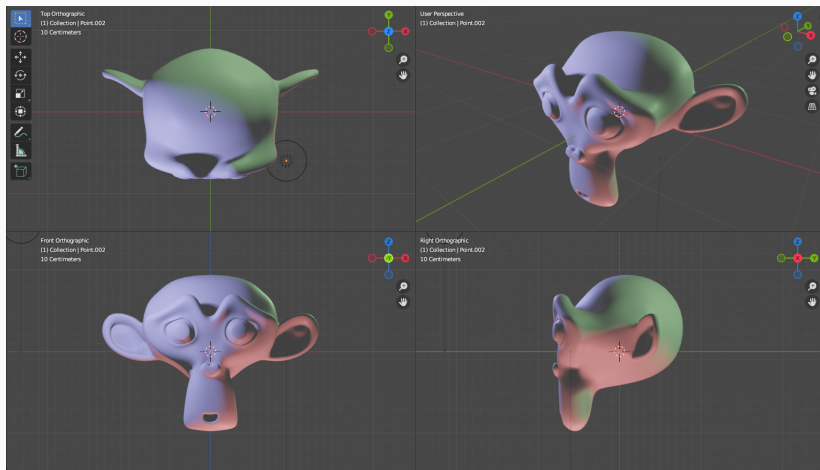
- 2D рендеринг: 2D игры, UI, карты





Параллельная проекция: применение

- 2D рендеринг: 2D игры, UI, карты
- Проектирование моделей, зданий, деталей (вид сверху, вид сбоку, вид спереди)



Параллельная проекция: применение

- 2D рендеринг: 2D игры, UI, карты
- Проектирование моделей, зданий, деталей (вид сверху, вид сбоку, вид спереди)
- Стилизация: 3D мир с ортографической проекцией (напр. изометрическая проекция)





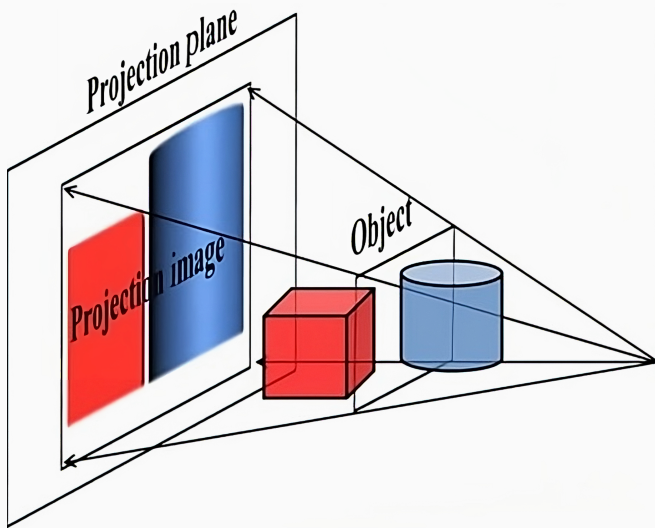
- Объекты на разном расстоянии от камеры выглядят одинаково

Параллельная проекция: проблемы

- Объекты на разном расстоянии от камеры выглядят одинаково
- Нельзя оценить расстояние до объекта по его изображению

- Объекты на разном расстоянии от камеры выглядят одинаково
- Нельзя оценить расстояние до объекта по его изображению
- Реальные камеры и глаза работают *не так*

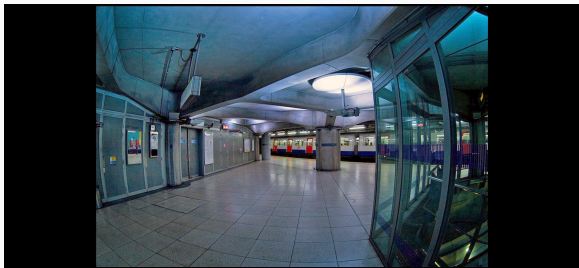
Перспективная проекция



- При перспективной проекции учитывается *расстояние* от объекта до камеры

- При перспективной проекции учитывается *расстояние* от объекта до камеры
- Два основных вида: *прямолинейная (ортоскопическая)* и *криволинейная*

Криволинейная vs прямолинейная перспектива



- При перспективной проекции учитывается *расстояние* от объекта до камеры
- Два основных вида: *прямолинейная (ортоскопическая)* и *криволинейная*
- В компьютерной графике всегда используют *прямолинейную*

- В рисовании известны *одноточечная*, *двухточечная*, и *трёхточечная* перспективы

- В рисовании известны *одноточечная*, *двухточечная*, и *трёхточечная* перспективы
- Это разновидности прямолинейной перспективы, отличающиеся тем, какие оси пространства или объекта параллельны плоскости камеры

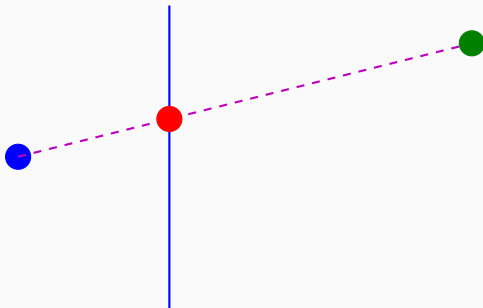
Перспективная проекция

- Есть центр проекции и плоскость проекции
-

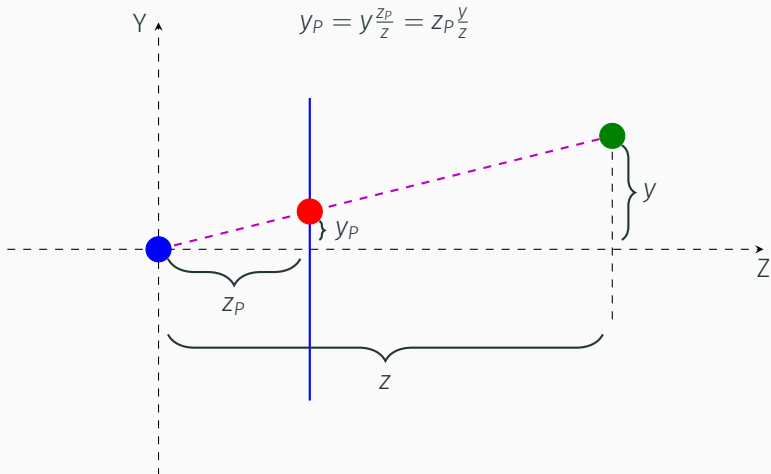


Перспективная проекция

- Есть центр проекции и плоскость проекции
- Проекция точки – пересечение прямой, проходящей через эту точку и центр проекции, с плоскостью проекции



Перспективная проекция



- Чтобы вычислить перспективную проекцию с центром в начале координат и плоскостью проекции $Z = z_p$, нужно разделить на Z -координату:

$$(x, y, z) \mapsto (z_p \frac{x}{z}, z_p \frac{y}{z})$$

- Чтобы вычислить перспективную проекцию с центром в начале координат и плоскостью проекции $Z = z_p$, нужно разделить на Z -координату:

$$(x, y, z) \mapsto (z_p \frac{x}{z}, z_p \frac{y}{z})$$

- Как выразить эту проекцию матрицей?

- Чтобы вычислить перспективную проекцию с центром в начале координат и плоскостью проекции $Z = z_p$, нужно разделить на Z -координату:

$$(x, y, z) \mapsto (z_p \frac{x}{z}, z_p \frac{y}{z})$$

- Как выразить эту проекцию матрицей? **Никак**: матрицы не позволяют делить одни координаты на другие (это нелинейная операция)

Perspective divide

- Чтобы поддержать перспективную проекцию, в графическом конвейере (после вершинного шейдера, перед переводом в экранные координаты) есть специальный *обязательный* шаг: **perspective divide**

Perspective divide

- Чтобы поддержать перспективную проекцию, в графическом конвейере (после вершинного шейдера, перед переводом в экранные координаты) есть специальный *обязательный* шаг: **perspective divide**
- `@builtin(position)` делится на последнюю координату `position.w`

$$(x, y, z, w) \mapsto \left(\frac{x}{w}, \frac{y}{w}, \frac{z}{w} \right)$$

Perspective divide

- Чтобы поддержать перспективную проекцию, в графическом конвейере (после вершинного шейдера, перед переводом в экранные координаты) есть специальный *обязательный* шаг: **perspective divide**
- `@builtin(position)` делится на последнюю координату `position.w`

$$(x, y, z, w) \mapsto \left(\frac{x}{w}, \frac{y}{w}, \frac{z}{w}\right)$$

- Если $w = 1$, ничего не меняется (ортографическая проекция)

Perspective divide

- Чтобы поддержать перспективную проекцию, в графическом конвейере (после вершинного шейдера, перед переводом в экранные координаты) есть специальный *обязательный* шаг: **perspective divide**
- `@builtin(position)` делится на последнюю координату `position.w`

$$(x, y, z, w) \mapsto \left(\frac{x}{w}, \frac{y}{w}, \frac{z}{w}\right)$$

- Если $w = 1$, ничего не меняется (ортографическая проекция)
- Если $w =$ расстояние до камеры, получается перспективная проекция

- Как выразить перспективную проекцию матрицей с последующим perspective divide?

- Как выразить перспективную проекцию матрицей с последующим perspective divide?

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} x \\ y \\ z \\ z \end{pmatrix} \mapsto \begin{pmatrix} x/z \\ y/z \\ 1 \end{pmatrix}$$

Перспективная проекция

- Как выразить перспективную проекцию матрицей с последующим perspective divide?

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} x \\ y \\ z \\ z \end{pmatrix} \mapsto \begin{pmatrix} x/z \\ y/z \\ 1 \end{pmatrix}$$

- Обычно матрица перспективной проекции выглядит **не так** из-за буфера глубины

- По умолчанию примитивы, рисуемые позже, накладываются поверх уже нарисованных

- По умолчанию примитивы, рисуемые позже, накладываются поверх уже нарисованных
- Обычно не проблема в 2D рендеринге: рисуем слои в нужном порядке

- По умолчанию примитивы, рисующиеся позже, накладываются поверх уже нарисованных
- Обычно не проблема в 2D рендеринге: рисуем слои в нужном порядке
 - 2D игра-платформер: фон, потом ландшафт, потом персонажи, потом эффекты

- По умолчанию примитивы, рисующиеся позже, накладываются поверх уже нарисованных
- Обычно не проблема в 2D рендеринге: рисуем слои в нужном порядке
 - 2D игра-платформер: фон, потом ландшафт, потом персонажи, потом эффекты
 - Карта: ландшафт, потом дороги и здания, потом иконки заведений

- По умолчанию примитивы, рисующиеся позже, накладываются поверх уже нарисованных
- Обычно не проблема в 2D рендеринге: рисуем слои в нужном порядке
 - 2D игра-платформер: фон, потом ландшафт, потом персонажи, потом эффекты
 - Карта: ландшафт, потом дороги и здания, потом иконки заведений
 - UI: фон виджета, потом рисунок кнопки, потом текст кнопки

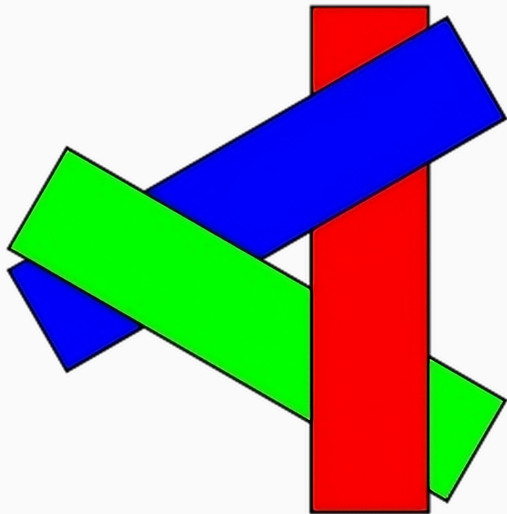
- В 3D можно сортировать треугольники в порядке убывания расстояния до камеры (painter's algorithm), но это плохое решение:

- В 3D можно сортировать треугольники в порядке убывания расстояния до камеры (painter's algorithm), но это плохое решение:
 - Нужно отсортировать миллионы треугольников каждый кадр и загрузить их на GPU $\implies$ хуже производительность

- В 3D можно сортировать треугольники в порядке убывания расстояния до камеры (painter's algorithm), но это плохое решение:
 - Нужно отсортировать миллионы треугольников каждый кадр и загрузить их на GPU $\implies$ хуже производительность
 - Треугольники одного объекта не идут непрерывно $\implies$ больше изменений состояния (переключений шейдеров, etc) при рендеринге $\implies$ хуже производительность

- В 3D можно сортировать треугольники в порядке убывания расстояния до камеры (painter's algorithm), но это плохое решение:
 - Нужно отсортировать миллионы треугольников каждый кадр и загрузить их на GPU $\implies$ хуже производительность
 - Треугольники одного объекта не идут непрерывно $\implies$ больше изменений состояния (переключений шейдеров, etc) при рендеринге $\implies$ хуже производительность
 - Всё равно решает проблему не во всех случаях (напр. когда в графе наложений объектов есть циклы, или треугольники пересекаются)

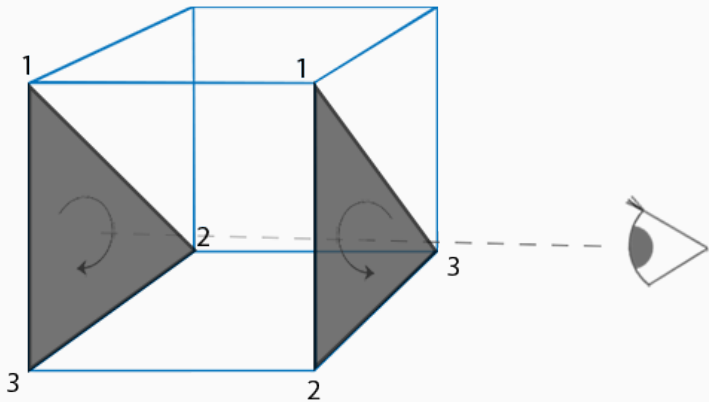
Случай, когда painter's algorithm не работает



- Другая идея: договоримся, что все треугольники описаны в порядке *против часовой стрелки*, если смотреть на модель *снаружи*

- Другая идея: договоримся, что все треугольники описаны в порядке *против часовой стрелки*, если смотреть на модель *снаружи*
- Тогда можно не рисовать треугольники, оказавшиеся на экране *по часовой стрелке* – они будут скрыты другими треугольниками ближе к камере

Back-face culling



- Другая идея: договоримся, что все треугольники описаны в порядке *против часовой стрелки*, если смотреть на модель *снаружи*
- Тогда можно не рисовать треугольники, оказавшиеся на экране *по часовой стрелке* – они будут скрыты другими треугольниками ближе к камере

Back-face culling

- Другая идея: договоримся, что все треугольники описаны в порядке *против часовой стрелки*, если смотреть на модель *снаружи*
- Тогда можно не рисовать треугольники, оказавшиеся на экране *по часовой стрелке* – они будут скрыты другими треугольниками ближе к камере
 - Из-за этого в некоторых играх вы видите объекты насквозь, когда проваливаетесь в них

Back-face culling

- Другая идея: договоримся, что все треугольники описаны в порядке *против часовой стрелки*, если смотреть на модель *снаружи*
- Тогда можно не рисовать треугольники, оказавшиеся на экране *по часовой стрелке* – они будут скрыты другими треугольниками ближе к камере
 - Из-за этого в некоторых играх вы видите объекты насквозь, когда проваливаетесь в них
- Работает только для сплошных 3D моделей без “дырок”, имеющих внутренность (напр. *manifold mesh*)

Back-face culling

- Другая идея: договоримся, что все треугольники описаны в порядке *против часовой стрелки*, если смотреть на модель *снаружи*
- Тогда можно не рисовать треугольники, оказавшиеся на экране *по часовой стрелке* – они будут скрыты другими треугольниками ближе к камере
 - Из-за этого в некоторых играх вы видите объекты насквозь, когда проваливаетесь в них
- Работает только для сплошных 3D моделей без “дырок”, имеющих внутренность (напр. *manifold mesh*)
- Решает проблему наложения только для выпуклых моделей

- Выключен по умолчанию

Back-face culling

- Выключен по умолчанию
- Настроить, какие грани отсекаются –
`renderPipelineDescriptor.primitive.cullFace`: МОЖЕТ БЫТЬ
None, Front, или Back

Back-face culling

- Выключен по умолчанию
- Настроить, какие грани отсекаются –
`renderPipelineDescriptor.primitive.cullFace`: может быть `None`, `Front`, или `Back`
- Настроить, какие треугольники считаются `Front`, а какие `Back`
– `renderPipelineDescriptor.primitive.cullFace`: может быть `CW` bkb `CCW`

- Стандарт индустрии: Front – это CCW (против часовой стрелки)

Back-face culling

- Стандарт индустрии: Front – это CCW (против часовой стрелки)
- Лучше всего его *никогда не менять*: большинство форматов и инструментов следуют этой договорённости

Back-face culling

- Стандарт индустрии: Front – это CCW (против часовой стрелки)
- Лучше всего его *никогда не менять*: большинство форматов и инструментов следуют этой договорённости
- Порядок вершин в `TriangleStrip` чётко определён, чтобы сохранять ориентацию вершин

Back-face culling

- Стандарт индустрии: Front – это CCW (против часовой стрелки)
- Лучше всего его *никогда не менять*: большинство форматов и инструментов следуют этой договорённости
- Порядок вершин в **TriangleStrip** чётко определён, чтобы сохранять ориентацию вершин
- Имеет смысл включать всегда, когда это возможно: перед этапом растеризации отсекается где-то половина треугольников

- Общее решение проблемы наложения объектов: *буфер глубины* (z-buffer)

- Общее решение проблемы наложения объектов: *буфер глубины* (z-buffer)
- Одноцветная текстура (изображение) такого же размера, как экран/окно

- Общее решение проблемы наложения объектов: *буфер глубины* (z-buffer)
- Одноцветная текстура (изображение) такого же размера, как экран/окно
- Его нужно создать самим и привязать к `render_pass`'у

- Общее решение проблемы наложения объектов: *буфер глубины* (z-buffer)
- Одноцветная текстура (изображение) такого же размера, как экран/окно
- Его нужно создать самим и привязать к `render_pass'y`
- Каждый пиксель хранит расстояние от нарисованного на экране пикселя до камеры





- Тест глубины (depth test): при рисовании очередного пикселя (*после* фрагментного шейдера) проверим: если уже нарисованный пиксель ближе к камере, то новый рисовать не будем

- Тест глубины (depth test): при рисовании очередного пикселя (*после* фрагментного шейдера) проверим: если уже нарисованный пиксель ближе к камере, то новый рисовать не будем
- Настраивается в `renderPipelineDescriptor.depthStencil`

- Тест глубины (depth test): при рисовании очередного пикселя (*после* фрагментного шейдера) проверим: если уже нарисованный пиксель ближе к камере, то новый рисовать не будем
- Настраивается в `renderPipelineDescriptor.depthStencil`
- Основная настройка – `depthCompare`: может принимать значения `Never`, `Always`, `Equal`, `NotEqual`, `Less`, `LessEqual`, `Greater`, `GreaterEqual`

- Тест глубины (depth test): при рисовании очередного пикселя (*после* фрагментного шейдера) проверим: если уже нарисованный пиксель ближе к камере, то новый рисовать не будем
- Настраивается в `renderPipelineDescriptor.depthStencil`
- Основная настройка – `depthCompare`: может принимать значения `Never`, `Always`, `Equal`, `NotEqual`, `Less`, `LessEqual`, `Greater`, `GreaterEqual`
- Например, `Less` означает, что пиксель нарисуеться, если его расстояние до камеры меньше расстояния, уже записанного в z-буфер

- Z-буфер хранит значения в формате 16/24-bit unsigned normalized или 32-bit floating-point

- Z-буфер хранит значения в формате 16/24-bit unsigned normalized или 32-bit floating-point
 - Обычно unsigned normalized 24-bit
 - Такой формат называется **Depth24Plus**, потому что хранится вместе со *stencil buffer*'ом, о нём поговорим позже

- Z-буфер хранит значения в формате 16/24-bit unsigned normalized или 32-bit floating-point
 - Обычно unsigned normalized 24-bit
 - Такой формат называется **Depth24Plus**, потому что хранится вместе со *stencil buffer*'ом, о нём поговорим позже
- Хранит значение координаты Z после perspective divide

- Z-буфер хранит значения в формате 16/24-bit unsigned normalized или 32-bit floating-point
 - Обычно unsigned normalized 24-bit
 - Такой формат называется **Depth24Plus**, потому что хранится вместе со *stencil buffer*'ом, о нём поговорим позже
- Хранит значение координаты Z после perspective divide
 - Доступно во фрагментном шейдере: `position.z`

- Z-буфер хранит значения в формате 16/24-bit unsigned normalized или 32-bit floating-point
 - Обычно unsigned normalized 24-bit
 - Такой формат называется **Depth24Plus**, потому что хранится вместе со *stencil buffer*'ом, о нём поговорим позже
- Хранит значение координаты Z после perspective divide
 - Доступно во фрагментном шейдере: `position.z`
 - Интервал $[0, 1]$ можно заменить с помощью `render_pass.set_viewport()`

- Z-буфер хранит значения в формате 16/24-bit unsigned normalized или 32-bit floating-point
 - Обычно unsigned normalized 24-bit
 - Такой формат называется **Depth24Plus**, потому что хранится вместе со *stencil buffer*'ом, о нём поговорим позже
- Хранит значение координаты Z после perspective divide
 - Доступно во фрагментном шейдере: `position.z`
 - Интервал $[0, 1]$ можно заменить с помощью `render_pass.set_viewport()`
 - В OpenGL диапазон глубин $[-1, 1]$, и к Z-координате после perspective divide дополнительно применяется преобразование $z \mapsto \frac{z+1}{2}$ перед записью в Z-буфер

- Фрагментный шейдер может изменить глубину фрагмента, записав новую глубину в `@builtin(frag_depth)`

Z-буфер: early depth test

- Фрагментный шейдер может изменить глубину фрагмента, записав новую глубину в `@builtin(frag_depth)`
- Это почти никогда не нужно и выключает *early depth test* – оптимизацию, при которой depth test происходит до фрагментного шейдера, а не после, и пропускает шейдер, если фрагмент всё равно не попадёт на экран

- Нужно очищать перед каждым новым кадром (настраивается в `renderPassDescriptor.depthStencilAttachment`)

- Нужно очищать перед каждым новым кадром (настраивается в `renderPassDescriptor.depthStencilAttachment`)
- Нужно обновлять размер при изменении размеров экрана

- Нужно очищать перед каждым новым кадром (настраивается в `renderPassDescriptor.depthStencilAttachment`)
- Нужно обновлять размер при изменении размеров экрана
- Можно использовать с любой проекцией (ортографической, перспективной)

- Нужно очищать перед каждым новым кадром (настраивается в `renderPassDescriptor.depthStencilAttachment`)
- Нужно обновлять размер при изменении размеров экрана
- Можно использовать с любой проекцией (ортографической, перспективной)
- Можно использовать и для рисования в 2D

- Z-координата (после perspective divide) должна быть в диапазоне $[0, 1]$

Depth clipping/clamping

- Z-координата (после perspective divide) должна быть в диапазоне $[0, 1]$
- Что делать с вершинами, выпадающими за диапазон? Два варианта:

Depth clipping/clamping

- Z-координата (после perspective divide) должна быть в диапазоне $[0, 1]$
- Что делать с вершинами, выпадающими за диапазон? Два варианта:
 - Depth clipping (отсечение по глубине, по умолчанию): примитив (точка/линия/треугольник) обрезается по плоскостям $Z = 0$ и $Z = 1$ (как будто вне этой области ничего нет)

Depth clipping/clamping

- Z-координата (после perspective divide) должна быть в диапазоне $[0, 1]$
- Что делать с вершинами, выпадающими за диапазон? Два варианта:
 - Depth clipping (отсечение по глубине, по умолчанию): примитив (точка/линия/треугольник) обрезается по плоскостям $Z = 0$ и $Z = 1$ (как будто вне этой области ничего нет)
 - Depth clamping: отсечения не происходит, но после растеризации глубина всех пикселей, не попавших в диапазон $[0, 1]$, сжимается до $[0, 1]$

Depth clipping/clamping

- Z-координата (после perspective divide) должна быть в диапазоне $[0, 1]$
- Что делать с вершинами, выпадающими за диапазон? Два варианта:
 - Depth clipping (отсечение по глубине, по умолчанию): примитив (точка/линия/треугольник) обрезается по плоскостям $Z = 0$ и $Z = 1$ (как будто вне этой области ничего нет)
 - Depth clamping: отсечения не происходит, но после растеризации глубина всех пикселей, не попавших в диапазон $[0, 1]$, сжимается до $[0, 1]$
 - WebGPU поддерживает depth clamping через расширение `depth-clip-control` и настройку `renderPipelineDescriptor.primitiveState.unclippedDepth`

- На самом деле, с координатами X и Y тоже происходит отсечение

Clipping

- На самом деле, с координатами X и Y тоже происходит отсечение
- Как будто пиксели, не попадающие на экран, просто игнорируются

Clipping

- На самом деле, с координатами X и Y тоже происходит отсечение
- Как будто пиксели, не попадающие на экран, просто игнорируются
- Не настраивается

Clipping

- На самом деле, с координатами X и Y тоже происходит отсечение
- Как будто пиксели, не попадающие на экран, просто игнорируются
- Не настраивается

$$-1 \leq X \leq 1$$

$$-1 \leq Y \leq 1$$

$$0 \leq Z \leq 1$$

Clipping

- На самом деле, с координатами X и Y тоже происходит отсечение
- Как будто пиксели, не попадающие на экран, просто игнорируются
- Не настраивается

$$-1 \leq X \leq 1$$

$$-1 \leq Y \leq 1$$

$$0 \leq Z \leq 1$$

- С учётом perspective divide (если $W > 0$):

$$-1 \leq X/W \leq 1 \quad -W \leq X \leq W$$

$$-1 \leq Y/W \leq 1 \Leftrightarrow -W \leq Y \leq W$$

$$0 \leq Z/W \leq 1 \quad 0 \leq Z \leq W$$

Clipping

- На самом деле, с координатами X и Y тоже происходит отсечение
- Как будто пиксели, не попадающие на экран, просто игнорируются
- Не настраивается

$$-1 \leq X \leq 1$$

$$-1 \leq Y \leq 1$$

$$0 \leq Z \leq 1$$

- С учётом perspective divide (если $W > 0$):

$$-1 \leq X/W \leq 1 \quad -W \leq X \leq W$$

$$-1 \leq Y/W \leq 1 \Leftrightarrow -W \leq Y \leq W$$

$$0 \leq Z/W \leq 1 \quad 0 \leq Z \leq W$$

- Если $W < 0$, точка **не попадает** на экран

Полный путь координат

- Входные данные: точка q_{object} в системе координат объекта/3D модели (object/model space)

Полный путь координат

- Входные данные: точка q_{object} в системе координат объекта/3D модели (object/model space)
- Позиционируем объект: $q_{world} = \text{Transform} \cdot q_{object}$ в мировых координатах (world space)

Полный путь координат

- Входные данные: точка q_{object} в системе координат объекта/3D модели (object/model space)
- Позиционируем объект: $q_{world} = \text{Transform} \cdot q_{object}$ в мировых координатах (world space)
- Учитываем расположение камеры: $q_{view} = \text{View} \cdot q_{world}$ в координатах камеры (view/camera space)

Полный путь координат

- Входные данные: точка q_{object} в системе координат объекта/3D модели (object/model space)
- Позиционируем объект: $q_{world} = \text{Transform} \cdot q_{object}$ в мировых координатах (world space)
- Учитываем расположение камеры: $q_{view} = \text{View} \cdot q_{world}$ в координатах камеры (view/camera space)
- Проекция камеры: `abuiltin(position)` = $q_{clip} = \text{Projection} \cdot q_{view}$ в clip space

Полный путь координат

- Входные данные: точка q_{object} в системе координат объекта/3D модели (object/model space)
- Позиционируем объект: $q_{world} = \text{Transform} \cdot q_{object}$ в мировых координатах (world space)
- Учитываем расположение камеры: $q_{view} = \text{View} \cdot q_{world}$ в координатах камеры (view/camera space)
- Проекция камеры: `abuiltin(position)` = $q_{clip} = \text{Projection} \cdot q_{view}$ в clip space
- Perspective divide: $q_{ndc} = xyz_{clip}/w_{clip}$ в normalized device coordinates

Полный путь координат

- Входные данные: точка q_{object} в системе координат объекта/3D модели (object/model space)
- Позиционируем объект: $q_{world} = \text{Transform} \cdot q_{object}$ в мировых координатах (world space)
- Учитываем расположение камеры: $q_{view} = \text{View} \cdot q_{world}$ в координатах камеры (view/camera space)
- Проекция камеры: `abuiltin(position)` = $q_{clip} = \text{Projection} \cdot q_{view}$ в clip space
- Perspective divide: $q_{ndc} = xyz_{clip}/w_{clip}$ в normalized device coordinates
 - x_{ndc} и y_{ndc} определяют координату пикселя на экране (через `render_pass.set_viewport()`)
 - z_{ndc} определяет глубину

Перспективная проекция: near & far clip planes

- Если записывать в w координату z , то после perspective divide $z = 1$, т.е. у всех точек одинаковая глубина, и не работает тест глубины $\implies$ нам нужна другая формула проекции!

Перспективная проекция: near & far clip planes

- Если записывать в w координату z , то после perspective divide $z = 1$, т.е. у всех точек одинаковая глубина, и не работает тест глубины $\implies$ нам нужна другая формула проекции!
- Итоговый диапазон глубин ограничен $\implies$ нужно как-то ограничить диапазон возможных расстояний до камеры

Перспективная проекция: near & far clip planes

- Если записывать в w координату z , то после perspective divide $z = 1$, т.е. у всех точек одинаковая глубина, и не работает тест глубины $\implies$ нам нужна другая формула проекции!
- Итоговый диапазон глубин ограничен $\implies$ нужно как-то ограничить диапазон возможных расстояний до камеры
- $\implies$ *Near* и *far* clip planes: $0 < \text{near} < \text{far} < \infty$

Перспективная проекция: near & far clip planes

- Если записывать в w координату z , то после perspective divide $z = 1$, т.е. у всех точек одинаковая глубина, и не работает тест глубины $\implies$ нам нужна другая формула проекции!
- Итоговый диапазон глубин ограничен $\implies$ нужно как-то ограничить диапазон возможных расстояний до камеры
- $\implies$ *Near* и *far* clip planes: $0 < near < far < \infty$
- Хотим, чтобы диапазон $z \in [near, far]$ после применения матрицы проекции и perspective divide превратился в $[0, 1]$

Перспективная проекция: near & far clip planes

- Хотим, чтобы диапазон $z \in [near, far]$ после применения матрицы проекции и perspective divide превратился в $[0, 1]$

Перспективная проекция: near & far clip planes

- Хотим, чтобы диапазон $z \in [near, far]$ после применения матрицы проекции и perspective divide превратился в $[0, 1]$
- Матрицей можно применить к z линейную функцию $Az + B$:

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & A & B \\ 0 & 0 & 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} x \\ y \\ Az + B \\ z \end{pmatrix}$$

Перспективная проекция: near & far clip planes

- Хотим, чтобы диапазон $z \in [near, far]$ после применения матрицы проекции и perspective divide превратился в $[0, 1]$
- Матрицей можно применить к z линейную функцию $Az + B$:

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & A & B \\ 0 & 0 & 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} x \\ y \\ Az + B \\ z \end{pmatrix}$$

- С учётом perspective divide:

$$z \mapsto \frac{Az+B}{z}$$

- С учётом perspective divide:

$$Z \mapsto \frac{Az+B}{Z}$$

Перспективная проекция: near & far clip planes

- С учётом perspective divide:

$$z \mapsto \frac{Az+B}{z}$$

$$near \mapsto \frac{A \cdot near + B}{near} = 0$$

$$far \mapsto \frac{A \cdot far + B}{far} = 1$$

Перспективная проекция: near & far clip planes

- С учётом perspective divide:

$$Z \mapsto \frac{Az+B}{Z}$$

$$near \mapsto \frac{A \cdot near + B}{near} = 0$$

$$far \mapsto \frac{A \cdot far + B}{far} = 1$$

- Линейная система на A и B:

$$A \cdot near + B = 0 \qquad A = \frac{far}{far - near}$$

$\implies$

$$A \cdot far + B = far \qquad B = -\frac{near \cdot far}{far - near}$$

Перспективная проекция: матрица проекции

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & A & B \\ 0 & 0 & 1 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & \frac{far}{far-near} & -\frac{near \cdot far}{far-near} \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

Перспективная проекция: правая система координат

- Обычно направление взгляда камеры выбирают не в сторону положительной оси Z , а в сторону отрицательной оси Z , чтобы получилась правая система координат

Перспективная проекция: правая система координат

- Обычно направление взгляда камеры выбирают не в сторону положительной оси Z , а в сторону отрицательной оси Z , чтобы получилась правая система координат
- Отсечение по $z = -near$ и $z = -far$

Перспективная проекция: правая система координат

- Обычно направление взгляда камеры выбирают не в сторону положительной оси Z , а в сторону отрицательной оси Z , чтобы получилась правая система координат
- Отсечение по $z = -near$ и $z = -far$
- Расстояние до камеры – не Z , а $-Z$

Перспективная проекция: правая система координат

- Обычно направление взгляда камеры выбирают не в сторону положительной оси Z , а в сторону отрицательной оси Z , чтобы получилась правая система координат
- Отсечение по $z = -near$ и $z = -far$
- Расстояние до камеры – не Z , а $-Z$

$$\frac{A(-near)+B}{near} = 0$$

$$\frac{A(-far)+B}{far} = 1$$

Перспективная проекция: правая система координат

- Обычно направление взгляда камеры выбирают не в сторону положительной оси Z , а в сторону отрицательной оси Z , чтобы получилась правая система координат
- Отсечение по $z = -near$ и $z = -far$
- Расстояние до камеры – не Z , а $-Z$

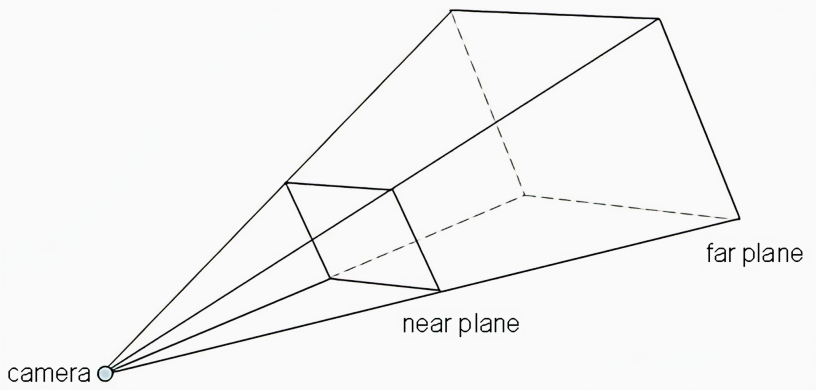
$$\frac{A(-near)+B}{near} = 0$$

$$\frac{A(-far)+B}{far} = 1$$

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & A & B \\ 0 & 0 & -1 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & \frac{far}{near-far} & \frac{near \cdot far}{near-far} \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

Перспективная проекция: view frustum

Область, видимая перспективной камерой – усечённая пирамида (frustum)



Как управлять шириной и высотой видимой области?

Как управлять шириной и высотой видимой области?

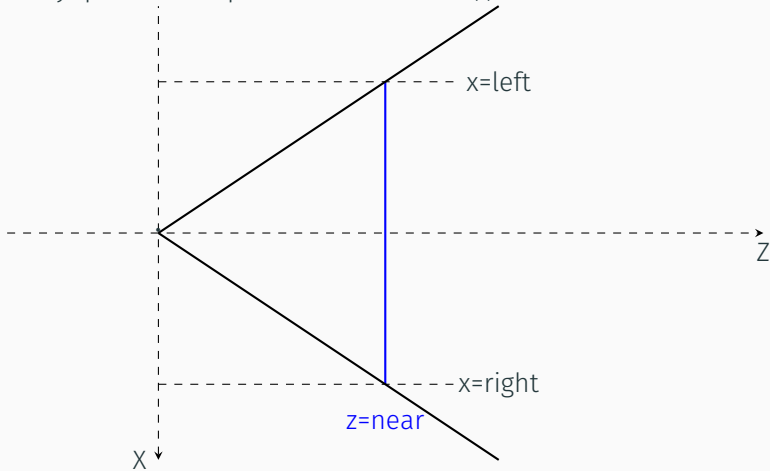
$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & A & B \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

Как управлять шириной и высотой видимой области?

$$\begin{pmatrix} C & 0 & D & 0 \\ 0 & E & F & 0 \\ 0 & 0 & A & B \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

Перспективная проекция: view frustum

Как управлять шириной и высотой видимой области?



Перспективная проекция: view frustum

Как управлять шириной и высотой видимой области?

- Обычно $left = -right$

Перспективная проекция: view frustum

Как управлять шириной и высотой видимой области?

- Обычно $left = -right$
- $\frac{-left}{near}$ – тангенс угла между осью (Z) проекции и левой границей видимой области
- $\frac{right}{near}$ – тангенс угла между осью (Z) проекции и правой границей видимой области

Перспективная проекция: view frustum

Как управлять шириной и высотой видимой области?

- Обычно $left = -right$
- $\frac{-left}{near}$ – тангенс угла между осью (Z) проекции и левой границей видимой области
- $\frac{right}{near}$ – тангенс угла между осью (Z) проекции и правой границей видимой области
- Аналогично top и $bottom$ для Y

Перспективная проекция: view frustum

Как управлять шириной и высотой видимой области?

- Обычно $left = -right$
- $\frac{-left}{near}$ – тангенс угла между осью (Z) проекции и левой границей видимой области
- $\frac{right}{near}$ – тангенс угла между осью (Z) проекции и правой границей видимой области
- Аналогично top и $bottom$ для Y
- Хотим, чтобы точка с координатами $X = left$ и $Z = -near$ перешла (после применения матрицы проекции и perspective divide) в точку с $X = -1$ и $Z = 0$
- Хотим, чтобы точка с координатами $X = right$ и $Z = -near$ перешла (после применения матрицы проекции и perspective divide) в точку с $X = 1$ и $Z = 0$

Перспективная проекция: view frustum

$$\begin{aligned} \frac{C \cdot \text{left} + D(-\text{near})}{\text{near}} &= -1 & C &= \frac{2 \cdot \text{near}}{\text{right} - \text{left}} \\ \Rightarrow \\ \frac{C \cdot \text{right} + D(-\text{near})}{\text{near}} &= 1 & D &= \frac{\text{right} + \text{left}}{\text{right} - \text{left}} \end{aligned}$$

Перспективная проекция: view frustum

$$\begin{aligned}\frac{C \cdot \text{left} + D(-\text{near})}{\text{near}} &= -1 & C &= \frac{2 \cdot \text{near}}{\text{right} - \text{left}} \\ \Rightarrow \\ \frac{C \cdot \text{right} + D(-\text{near})}{\text{near}} &= 1 & D &= \frac{\text{right} + \text{left}}{\text{right} - \text{left}}\end{aligned}$$

Аналогично для Y

$$\begin{aligned}\frac{E \cdot \text{bottom} + F(-\text{near})}{\text{near}} &= -1 & E &= \frac{2 \cdot \text{near}}{\text{top} - \text{bottom}} \\ \Rightarrow \\ \frac{E \cdot \text{top} + F(-\text{near})}{\text{near}} &= 1 & F &= \frac{\text{top} + \text{bottom}}{\text{top} - \text{bottom}}\end{aligned}$$

Перспективная проекция: матрица проекции

$$\begin{pmatrix} \frac{2 \cdot \text{near}}{\text{right} - \text{left}} & 0 & \frac{\text{right} + \text{left}}{\text{right} - \text{left}} & 0 \\ 0 & \frac{2 \cdot \text{near}}{\text{top} - \text{bottom}} & \frac{\text{top} + \text{bottom}}{\text{top} - \text{bottom}} & 0 \\ 0 & 0 & \frac{\text{far}}{\text{near} - \text{far}} & \frac{\text{near} \cdot \text{far}}{\text{near} - \text{far}} \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

Перспективная проекция: матрица проекции

$$\begin{pmatrix} \frac{2 \cdot \text{near}}{\text{right} - \text{left}} & 0 & \frac{\text{right} + \text{left}}{\text{right} - \text{left}} & 0 \\ 0 & \frac{2 \cdot \text{near}}{\text{top} - \text{bottom}} & \frac{\text{top} + \text{bottom}}{\text{top} - \text{bottom}} & 0 \\ 0 & 0 & \frac{\text{far}}{\text{near} - \text{far}} & \frac{\text{near} \cdot \text{far}}{\text{near} - \text{far}} \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

Часто $\text{left} = -\text{right}$ и $\text{bottom} = -\text{top}$, тогда

$$\begin{pmatrix} \frac{\text{near}}{\text{right}} & 0 & 0 & 0 \\ 0 & \frac{\text{near}}{\text{top}} & 0 & 0 \\ 0 & 0 & \frac{\text{far}}{\text{near} - \text{far}} & \frac{\text{near} \cdot \text{far}}{\text{near} - \text{far}} \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

- Это самый часто встречающийся вид матрицы перспективной проекции

Перспективная проекция: матрица проекции

$$\begin{pmatrix} \frac{2 \cdot \text{near}}{\text{right} - \text{left}} & 0 & \frac{\text{right} + \text{left}}{\text{right} - \text{left}} & 0 \\ 0 & \frac{2 \cdot \text{near}}{\text{top} - \text{bottom}} & \frac{\text{top} + \text{bottom}}{\text{top} - \text{bottom}} & 0 \\ 0 & 0 & \frac{\text{far}}{\text{near} - \text{far}} & \frac{\text{near} \cdot \text{far}}{\text{near} - \text{far}} \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

Часто $\text{left} = -\text{right}$ и $\text{bottom} = -\text{top}$, тогда

$$\begin{pmatrix} \frac{\text{near}}{\text{right}} & 0 & 0 & 0 \\ 0 & \frac{\text{near}}{\text{top}} & 0 & 0 \\ 0 & 0 & \frac{\text{far}}{\text{near} - \text{far}} & \frac{\text{near} \cdot \text{far}}{\text{near} - \text{far}} \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

- Это самый часто встречающийся вид матрицы перспективной проекции
- **NB:** в OpenGL матрица немного отличается из-за другого диапазона глубин $[-1, 1]$

Перспективная проекция: матрица проекции

$$\begin{pmatrix} \frac{2 \cdot \text{near}}{\text{right} - \text{left}} & 0 & \frac{\text{right} + \text{left}}{\text{right} - \text{left}} & 0 \\ 0 & \frac{2 \cdot \text{near}}{\text{top} - \text{bottom}} & \frac{\text{top} + \text{bottom}}{\text{top} - \text{bottom}} & 0 \\ 0 & 0 & \frac{\text{far}}{\text{near} - \text{far}} & \frac{\text{near} \cdot \text{far}}{\text{near} - \text{far}} \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

Часто $\text{left} = -\text{right}$ и $\text{bottom} = -\text{top}$, тогда

$$\begin{pmatrix} \frac{\text{near}}{\text{right}} & 0 & 0 & 0 \\ 0 & \frac{\text{near}}{\text{top}} & 0 & 0 \\ 0 & 0 & \frac{\text{far}}{\text{near} - \text{far}} & \frac{\text{near} \cdot \text{far}}{\text{near} - \text{far}} \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

- Это самый часто встречающийся вид матрицы перспективной проекции
- **NB:** в OpenGL матрица немного отличается из-за другого диапазона глубин $[-1, 1]$
- Эту матрицу мы передаём в шейдер и применяем к входным вершинам (после других преобразований)

- Как выбирать параметры камеры?

- Как выбирать параметры камеры?
- Пусть хотим камеру с шириной обзора по X равной θ радиан, и с *aspect ratio* (отношение ширины к высоте) равным R

- Как выбирать параметры камеры?
- Пусть хотим камеру с шириной обзора по X равной θ радиан, и с *aspect ratio* (отношение ширины к высоте) равным R
- $-left = right = near \cdot \tan\left(\frac{\theta}{2}\right)$
- $-bottom = top = \frac{1}{R} \cdot right = \frac{near}{R} \cdot \tan\left(\frac{\theta}{2}\right)$

- Как выбирать параметры near и far?

- Как выбирать параметры near и far?
- Все объекты ближе near или дальше far обрезаются – почему не взять near= 0 и far= ∞ ?

- Как выбирать параметры near и far?
- Все объекты ближе near или дальше far обрезаются – почему не взять near= 0 и far= ∞ ?
- Если near = 0, третья строка матрицы проекции вырождается в $\begin{pmatrix} 0 & 0 & -1 & 0 \end{pmatrix}$, и у всех точек будет одинаковая глубина, т.е. не получится использовать z-буфер

Перспективная проекция

- Как выбирать параметры $near$ и far ?
- Все объекты ближе $near$ или дальше far обрезаются – почему не взять $near = 0$ и $far = \infty$?
- Если $near = 0$, третья строка матрицы проекции вырождается в $\begin{pmatrix} 0 & 0 & -1 & 0 \end{pmatrix}$, и у всех точек будет одинаковая глубина, т.е. не получится использовать z-буфер
- Если $far \rightarrow \infty$, третья строка стремится к $\begin{pmatrix} 0 & 0 & -1 & -2near \end{pmatrix}$, что само по себе ничего не ломает

- Z-буфер имеет ограниченную точность

- Z-буфер имеет ограниченную точность
- Чем больше отношение $\frac{far}{near}$, тем меньше точности приходится на единицу расстояния

- Z-буфер имеет ограниченную точность
- Чем больше отношение $\frac{far}{near}$, тем меньше точности приходится на единицу расстояния
- Как выбирать параметры near и far?

Перспективная проекция

- Z-буфер имеет ограниченную точность
- Чем больше отношение $\frac{far}{near}$, тем меньше точности приходится на единицу расстояния
- Как выбирать параметры near и far?
- near: максимально возможный, при котором не обрезается что-то существенное
 - При этом камера искусственно держится на некотором расстоянии от любых объектов

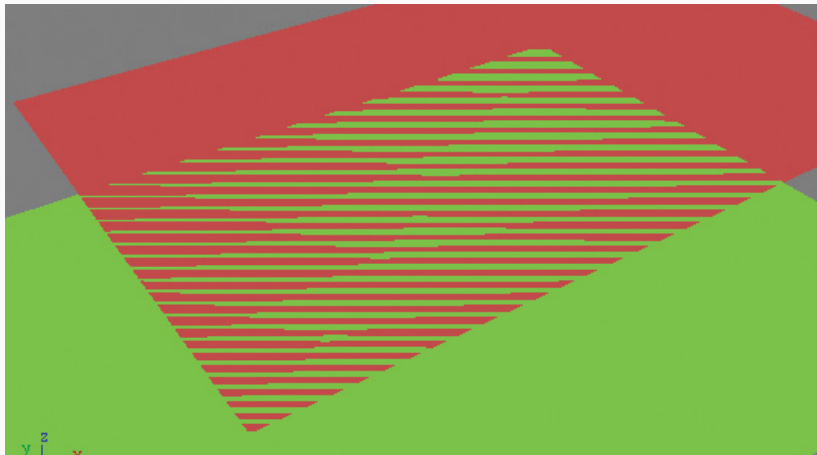
Перспективная проекция

- Z-буфер имеет ограниченную точность
- Чем больше отношение $\frac{far}{near}$, тем меньше точности приходится на единицу расстояния
- Как выбирать параметры near и far?
- near: максимально возможный, при котором не обрезается что-то существенное
 - При этом камера искусственно держится на некотором расстоянии от любых объектов
- far: минимально возможный, при котором видно всю сцену

- Z-буфер имеет ограниченную точность
- Чем больше отношение $\frac{far}{near}$, тем меньше точности приходится на единицу расстояния
- Как выбирать параметры near и far?
- near: максимально возможный, при котором не обрезается что-то существенное
 - При этом камера искусственно держится на некотором расстоянии от любых объектов
- far: минимально возможный, при котором видно всю сцену
- Нехватка точности z-буфера приводит к *z-fighting*'у

Z-fighting

Ситуация, когда две очень близких почти (или полностью) параллельных плоскости рисуются с артефактом из-за недостаточной точности Z-буфера:



Перспективная проекция: матрица проекции

- Что, если мы хотим переместить и повернуть камеру?

Перспективная проекция: матрица проекции

- Что, если мы хотим переместить и повернуть камеру?
- Так же, как с ортографической проекцией: сначала применим обратное преобразование, а потом матрицу проекции

Перспективная проекция: матрица проекции

- Что, если мы хотим переместить и повернуть камеру?
- Так же, как с ортографической проекцией: сначала применим обратное преобразование, а потом матрицу проекции

$$\begin{pmatrix} C & 0 & D & 0 \\ 0 & E & F & 0 \\ 0 & 0 & A & B \\ 0 & 0 & -1 & 0 \end{pmatrix} \cdot \begin{pmatrix} X_x & Y_x & Z_x & C_x \\ X_y & Y_y & Z_y & C_y \\ X_z & Y_z & Z_z & C_z \\ 0 & 0 & 0 & 1 \end{pmatrix}^{-1} \quad (1)$$

Projection · View

Перспективная проекция: матрица проекции

- Что, если мы хотим переместить и повернуть камеру?
- Так же, как с ортографической проекцией: сначала применим обратное преобразование, а потом матрицу проекции

$$\begin{pmatrix} C & 0 & D & 0 \\ 0 & E & F & 0 \\ 0 & 0 & A & B \\ 0 & 0 & -1 & 0 \end{pmatrix} \cdot \begin{pmatrix} X_x & Y_x & Z_x & C_x \\ X_y & Y_y & Z_y & C_y \\ X_z & Y_z & Z_z & C_z \\ 0 & 0 & 0 & 1 \end{pmatrix}^{-1} \quad (1)$$

Projection · View

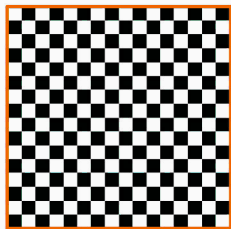
- Обычно именно эти матрицы (или их произведение) передают в шейдер

- songho.ca/opengl/gl_transform.html
- songho.ca/opengl/gl_projectionmatrix.html

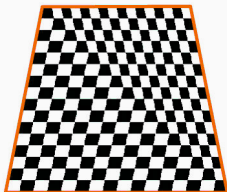
Perspective-correct интерполяция атрибутов

- Перспективная проекция портит интерполяцию атрибутов между вершинным и фрагментным шейдерами

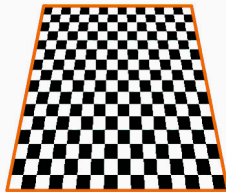
Perspective-correct интерполяция атрибутов



Texture



Affine
screen-space
interpolation



Perspective
world-space
interpolation

Perspective-correct интерполяция атрибутов

- Перспективная проекция портит интерполяцию атрибутов между вершинным и фрагментным шейдерами

Perspective-correct интерполяция атрибутов

- Перспективная проекция портит интерполяцию атрибутов между вершинным и фрагментным шейдерами
- На самом деле, барицентрические координаты вычисляются с учётом перспективной проекции:

Perspective-correct интерполяция атрибутов

- Перспективная проекция портит интерполяцию атрибутов между вершинным и фрагментным шейдерами
- На самом деле, барицентрические координаты вычисляются с учётом перспективной проекции:
 - Между вершинами линейно интерполируется $1 / \text{position.w}$ из вершинного шейдера

Perspective-correct интерполяция атрибутов

- Перспективная проекция портит интерполяцию атрибутов между вершинным и фрагментным шейдерами
- На самом деле, барицентрические координаты вычисляются с учётом перспективной проекции:
 - Между вершинами линейно интерполируется `1 / position.w` из вершинного шейдера
 - Исходные барицентрические координаты λ_i (посчитанные в экранных координатах) заменяются на

$$\frac{\lambda_i / w_i}{\sum \lambda_k / w_k}$$

Perspective-correct интерполяция атрибутов

- Перспективная проекция портит интерполяцию атрибутов между вершинным и фрагментным шейдерами
- На самом деле, барицентрические координаты вычисляются с учётом перспективной проекции:
 - Между вершинами линейно интерполируется `1 / position.w` из вершинного шейдера
 - Исходные барицентрические координаты λ_i (посчитанные в экранных координатах) заменяются на

$$\frac{\lambda_i/w_i}{\sum \lambda_k/w_k}$$

- Проинтерполированное значение $1/w$ доступно во фрагментном шейдере: `position.w`

Perspective-correct интерполяция атрибутов

- Перспективная проекция портит интерполяцию атрибутов между вершинным и фрагментным шейдерами
- На самом деле, барицентрические координаты вычисляются с учётом перспективной проекции:
 - Между вершинами линейно интерполируется `1 / position.w` из вершинного шейдера
 - Исходные барицентрические координаты λ_i (посчитанные в экранных координатах) заменяются на

$$\frac{\lambda_i/w_i}{\sum \lambda_k/w_k}$$

- Проинтерполированное значение $1/w$ доступно во фрагментном шейдере: `position.w`
- Можно выключить это поведение для конкретного атрибута с помощью модификатора `@interpolate(linear)` в шейдере

- `StackOverflow`: How exactly does OpenGL do perspective correct linear interpolation
- `Scratchapixel`: Perspective-correct interpolation of vertex attributes

Бонус: как найти координаты камеры, зная матрицу камеры?

- Пусть $T = \text{Projection} \cdot \text{View}$ – матрица камеры

Бонус: как найти координаты камеры, зная матрицу камеры?

- Пусть $T = \text{Projection} \cdot \text{View}$ – матрица камеры
- При ортографической проекции центр видимой области переходит в $(0, 0, 0, 1)$

Бонус: как найти координаты камеры, зная матрицу камеры?

- Пусть $T = \text{Projection} \cdot \text{View}$ – матрица камеры
- При ортографической проекции центр видимой области переходит в $(0, 0, 0, 1)$
- $\implies$ нужно вычислить $T^{-1} \cdot (0, 0, 0, 1)$

Бонус: как найти координаты камеры, зная матрицу камеры?

- Пусть $T = \text{Projection} \cdot \text{View}$ – матрица камеры
- При ортографической проекции центр видимой области переходит в $(0, 0, 0, 1)$
- $\implies$ нужно вычислить $T^{-1} \cdot (0, 0, 0, 1)$
- При перспективной проекции центр проекции переходит в бесконечно удалённую точку $(0, 0, -1, 0)$

Бонус: как найти координаты камеры, зная матрицу камеры?

- Пусть $T = \text{Projection} \cdot \text{View}$ – матрица камеры
- При ортографической проекции центр видимой области переходит в $(0, 0, 0, 1)$
- $\implies$ нужно вычислить $T^{-1} \cdot (0, 0, 0, 1)$
- При перспективной проекции центр проекции переходит в бесконечно удалённую точку $(0, 0, -1, 0)$
- $\implies$ нужно вычислить $T^{-1} \cdot (0, 0, -1, 0)$ и применить к результату perspective divide (разделить XYZ на W)

Бонус: как найти координаты камеры, зная матрицу камеры?

- Пусть $T = \text{Projection} \cdot \text{View}$ – матрица камеры
- При ортографической проекции центр видимой области переходит в $(0, 0, 0, 1)$
- $\implies$ нужно вычислить $T^{-1} \cdot (0, 0, 0, 1)$
- При перспективной проекции центр проекции переходит в бесконечно удалённую точку $(0, 0, -1, 0)$
- $\implies$ нужно вычислить $T^{-1} \cdot (0, 0, -1, 0)$ и применить к результату perspective divide (разделить XYZ на W)
- Полезно, когда позиция камеры не задаётся явно, а вычисляется на основе других величин (точка фокуса камеры, углы поворота, расстояние до фокуса, etc)

Бонус: как перевести из экранных координат в мировые?

- Есть точка на экране с координатами $(P_x, P_y) \in [-1, 1]^2$
(например, координаты указателя мыши, переведённые из пиксельных координат в NDC)

Бонус: как перевести из экранных координат в мировые?

- Есть точка на экране с координатами $(P_x, P_y) \in [-1, 1]^2$ (например, координаты указателя мыши, переведённые из пиксельных координат в NDC)
- В пространстве точке на экране соответствует луч из камеры – как его найти?

Бонус: как перевести из экранных координат в мировые?

- Есть точка на экране с координатами $(P_x, P_y) \in [-1, 1]^2$ (например, координаты указателя мыши, переведённые из пиксельных координат в NDC)
- В пространстве точке на экране соответствует луч из камеры – как его найти?
- Для перспективной проекции: найдём центр проекции, и точку на near или far плоскости, где её пересекает этот луч

Бонус: как перевести из экранных координат в мировые?

- Есть точка на экране с координатами $(P_x, P_y) \in [-1, 1]^2$ (например, координаты указателя мыши, переведённые из пиксельных координат в NDC)
- В пространстве точке на экране соответствует луч из камеры – как его найти?
- Для перспективной проекции: найдём центр проекции, и точку на near или far плоскости, где её пересекает этот луч
- Для любой проекции: найдём две точки, пересекающие этот луч, на near и far плоскостях ($P_z = 0$ и $P_z = 1$)

Бонус: как перевести из экранных координат в мировые?

- Пусть Proj – оператор, выполняющий perspective divide:

$$\text{Proj} \begin{pmatrix} x \\ y \\ z \\ w \end{pmatrix} = \begin{pmatrix} x/w \\ y/w \\ z/w \end{pmatrix}$$

Бонус: как перевести из экранных координат в мировые?

- Пусть Proj – оператор, выполняющий perspective divide:

$$\text{Proj} \begin{pmatrix} x \\ y \\ z \\ w \end{pmatrix} = \begin{pmatrix} x/w \\ y/w \\ z/w \end{pmatrix}$$

- Мы хотим решить уравнение

$$\text{Proj} \cdot T \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} p_x \\ p_y \\ p_z \end{pmatrix}$$

Бонус: как перевести из экранных координат в мировые?

- Мы хотим решить уравнение

$$\text{Proj} \cdot T \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} p_x \\ p_y \\ p_z \end{pmatrix}$$

Бонус: как перевести из экранных координат в мировые?

- Мы хотим решить уравнение

$$\text{Proj} \cdot T \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} p_x \\ p_y \\ p_z \end{pmatrix}$$

- Как выглядит множество $\text{Proj}^{-1} \begin{pmatrix} x \\ y \\ z \end{pmatrix}$?

Бонус: как перевести из экранных координат в мировые?

- Мы хотим решить уравнение

$$\text{Proj} \cdot T \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} p_x \\ p_y \\ p_z \end{pmatrix}$$

- Как выглядит множество $\text{Proj}^{-1} \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} \lambda x \\ \lambda y \\ \lambda z \\ \lambda \end{pmatrix} \forall \lambda \neq 0$

Бонус: как перевести из экранных координат в мировые?

- Мы хотим решить уравнение

$$\text{Proj} \cdot T \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} p_x \\ p_y \\ p_z \end{pmatrix}$$

- Как выглядит множество $\text{Proj}^{-1} \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} \lambda x \\ \lambda y \\ \lambda z \\ \lambda \end{pmatrix} \forall \lambda \neq 0$

$$\Rightarrow T \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} \lambda p_x \\ \lambda p_y \\ \lambda p_z \\ \lambda \end{pmatrix}, \text{ но мы не знаем } \lambda$$

Бонус: как перевести из экранных координат в мировые?

$$\begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = T^{-1} \begin{pmatrix} \lambda P_X \\ \lambda P_Y \\ \lambda P_Z \\ \lambda \end{pmatrix} = \lambda T^{-1} \begin{pmatrix} P_X \\ P_Y \\ P_Z \\ 1 \end{pmatrix}$$

Бонус: как перевести из экранных координат в мировые?

$$\begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = T^{-1} \begin{pmatrix} \lambda P_X \\ \lambda P_Y \\ \lambda P_Z \\ \lambda \end{pmatrix} = \lambda T^{-1} \begin{pmatrix} P_X \\ P_Y \\ P_Z \\ 1 \end{pmatrix}$$

- Приравняем последнюю координату левой и правой стороны уравнения:

$$1 = \lambda \cdot \left[T^{-1} \begin{pmatrix} P_X \\ P_Y \\ P_Z \\ 1 \end{pmatrix} \right]_w$$

Бонус: как перевести из экранных координат в мировые?

$$\lambda = \frac{1}{\left[T^{-1} \begin{pmatrix} P_X \\ P_Y \\ P_Z \\ 1 \end{pmatrix} \right]_w}$$

Бонус: как перевести из экранных координат в мировые?

$$\lambda = \frac{1}{\left[T^{-1} \begin{pmatrix} P_X \\ P_Y \\ P_Z \\ 1 \end{pmatrix} \right]_w}$$

$$\lambda T^{-1} \begin{pmatrix} P_X \\ P_Y \\ P_Z \\ 1 \end{pmatrix} = \frac{T^{-1} \begin{pmatrix} P_X \\ P_Y \\ P_Z \\ 1 \end{pmatrix}}{\left[T^{-1} \begin{pmatrix} P_X \\ P_Y \\ P_Z \\ 1 \end{pmatrix} \right]_w}$$

Бонус: как перевести из экранных координат в мировые?

- Вектор $T^{-1} \begin{pmatrix} P_x \\ P_y \\ P_z \\ 1 \end{pmatrix}$ нужно *разделить на последнюю координату!*

Бонус: как перевести из экранных координат в мировые?

- Вектор $T^{-1} \begin{pmatrix} P_X \\ P_Y \\ P_Z \\ 1 \end{pmatrix}$ нужно разделить на последнюю координату!

$$\begin{pmatrix} x \\ y \\ z \end{pmatrix} = \text{Proj } T^{-1} \begin{pmatrix} P_X \\ P_Y \\ P_Z \\ 1 \end{pmatrix}$$

Бонус: как перевести из экранных координат в мировые?

- Вектор $T^{-1} \begin{pmatrix} P_X \\ P_Y \\ P_Z \\ 1 \end{pmatrix}$ нужно разделить на последнюю координату!

$$\begin{pmatrix} x \\ y \\ z \end{pmatrix} = \text{Proj } T^{-1} \begin{pmatrix} P_X \\ P_Y \\ P_Z \\ 1 \end{pmatrix}$$

- Работает для любой проекции (и перспективной, и ортографической)

Бонус: как перевести из экранных координат в мировые?

$$\text{Proj } T \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} a \\ b \\ c \end{pmatrix}$$

$$\Rightarrow \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \text{Proj } T^{-1} \begin{pmatrix} a \\ b \\ c \\ 1 \end{pmatrix}$$

Бонус: как найти координаты вершин видимой области (view frustum)?

$$\begin{pmatrix} x \\ y \\ z \end{pmatrix} = \text{Proj } T^{-1} \begin{pmatrix} \pm 1 \\ \pm 1 \\ 0/1 \\ 1 \end{pmatrix}$$

- На больших сценах, или при больших координатах центра камеры, view-projection матрица очень плохо обусловлена, и её обращение даёт большие численные погрешности

Бонус: проблемы на больших сценах

- На больших сценах, или при больших координатах центра камеры, view-projection матрица очень плохо обусловлена, и её обращение даёт большие численные погрешности
- В таких случаях лучше пользоваться более прямыми методами (вычислять всё это из исходных параметров камеры)